

Temporal Video State Change Detection

Embedding-Driven Object State Tracking in Procedural Videos

Ansh Kanungo
Northeastern University
kanungo.an@northeastern.edu

Shravan Chandrachud
Northeastern University
chandrachud.s@northeastern.edu

April 2026

Abstract

We present a system for automatically detecting and logging object state changes in procedural cooking videos. The pipeline operates in two stages: (1) an embedding-based change detector that identifies visually significant keyframes, reducing the number of frames requiring classification by approximately 80%, and (2) a state classifier that assigns object-state labels to each flagged frame. We evaluate two embedding backbones (DINOv2-small and CLIP ViT-B/32) and five classification approaches spanning supervised learning (SVM with RBF and linear kernels, Logistic Regression), unsupervised deep neural network clustering (DEC), and zero-shot Vision-Language Models (Gemini 2.5 Flash with Gemma 4 E4B fallback). All methods are evaluated on a 90-second omelette-making video. The best supervised classifier (SVM RBF) achieves 85.5% test accuracy on the 12-class recognition task but produces a noisy memory bank with extensive state flickering (14 egg-state oscillations). The zero-shot VLM, leveraging autoregressive context and a forward-only constraint, produces a clean memory bank of 9 confirmed transitions with no training data and no flickering, additionally supporting multi-instance tracking and position-aware object identification.

GitHub: [Temporal Video State Change Detection](#)

1 Introduction

Procedural videos—instructional recordings of step-by-step tasks such as cooking, assembly, and repair—represent one of the fastest-growing categories of online content. Understanding *what happens* in these videos at the object level is critical for applications in robotic imitation learning [5], automated quality inspection, video summarization, and assistive technology.

The problem we address is **temporal object state change detection**: given a procedural video and a task description, automatically produce a structured log—a *memory bank*—of every meaningful state transition for every key object, annotated with timestamps and frame numbers. For example, in an omelette-making video, the system should detect that an egg transitions through `whole_in_shell` → `cracked` → `beaten` → `cooking` → `folded_omelette` → `plated`, recording the exact moment each transition occurs.

1.1 Challenges

- **Temporal redundancy.** A 90-second video at 30 FPS contains 2,702 frames. Sampling at 2 FPS reduces this to 193 frames, and our embedding-based change detection further filters these to ~30–40 flagged frames, yet meaningful state transitions occur at only 8–12 moments. Even with this ~80% reduction, classifying every flagged frame independently introduces noise due to state flickering.

- **Fine-grained visual distinction.** Many transitions involve subtle visual changes—a raw egg versus a beaten egg in the same bowl differs only in texture uniformity. Standard classifiers struggle with such intra-category variation.
- **State flickering.** Frame-level classifiers lack temporal context: they classify each frame independently, causing rapid oscillation between visually similar states (e.g., *whisked* $\leftrightarrow$ *omelette*) when the visual signal is ambiguous. This produces unusable memory banks.
- **Multi-instance disambiguation.** Real cooking scenarios involve multiple instances of the same class (e.g., two eggs at different lifecycle stages). The system must track each instance independently.
- **Zero-shot generalization.** Hardcoding states for every possible recipe is infeasible. An ideal system should generalize to new tasks without task-specific training data.

1.2 Applications

A reliable state tracker has direct applications in robotic learning from demonstration, automated recipe documentation, manufacturing quality control, and temporal video question answering (e.g., “When was the egg added to the pan?”).

2 System Overview

The pipeline consists of three stages (Figure 1):

Stage 1: Schema Generation. An LLM receives a task description and generates a JSON schema containing all relevant objects, their ordered visual states, and high-level process steps.

Stage 2: Change Detection. Video frames are sampled at 2 FPS and passed through a vision encoder to extract dense embeddings. Cosine similarity between each frame and a *rolling anchor* (the last frame where a state change was confirmed) is computed. Frames below a threshold τ are flagged, reducing classifier invocations by $\sim 80\%$.

Stage 3: State Classification. Flagged frames are processed by a classifier. We evaluate five classifiers: supervised models (SVM RBF, SVM Linear, Logistic Regression), an unsupervised neural network for clustering (DEC), and a zero-shot VLM.

Both paradigms share the same change detection front-end but use different embedding backbones: CLIP ViT-B/32 (512-dim) for the supervised classifiers and DINOv2-small (384-dim) for the VLM approach. CLIP’s text-aligned space suits classifiers mapping to discrete labels; DINOv2 provides superior visual discrimination for similarity-based change detection.

Classification vs. Regression. The pipeline combines both: Stage 2 is a *regression* task—cosine similarity produces a continuous score in $[0, 1]$, thresholded into a binary decision. Stage 3 is a *multi-class classification* task—each flagged frame is assigned to one of 12 discrete state labels.

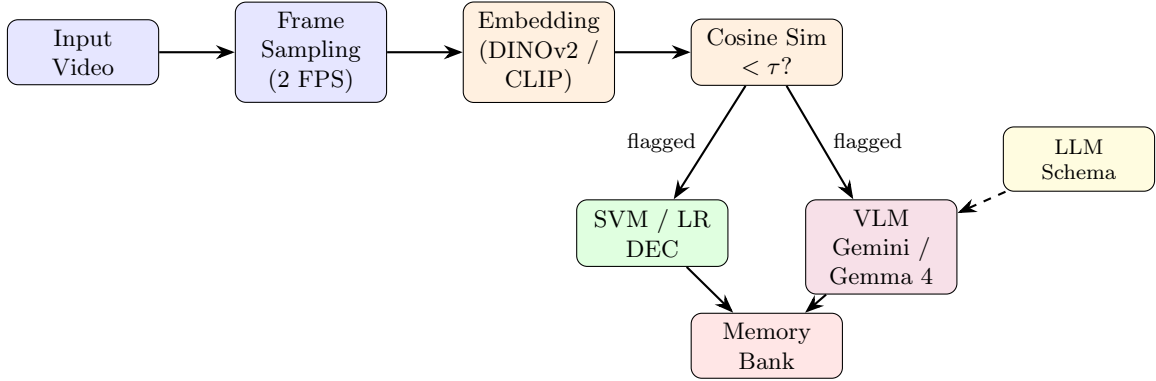


Figure 1: Pipeline architecture. Both classification paradigms share the embedding-based change detection front-end. The VLM branch receives an LLM-generated schema with process steps.

3 Dataset

A custom-curated dataset of 9,843 images across 7 kitchen objects and 12 active state classes was assembled for the omelette-making domain. One class (**butter/melted**) contains no images and is excluded. Images were sourced from Roboflow Universe, Kaggle, Mendeley Data, and the Pexels API. The split is approximately 78.5% train / 8.1% test / 9.3% val. Cross-validation used 5 folds on combined train+val. Table 1 provides the full breakdown.

Table 1: Dataset: 9,843 images across 7 objects and 12 active state classes.

Object	State	Train	Test	Val	Total
Egg	raw	1674	90	164	1928
	whisked	400	50	50	500
	omelette	1300	225	225	1750
	burned	400	50	50	500
Pan	empty	165	20	22	207
	with-food	400	50	50	500
Butter	solid	519	24	52	595
Oil	general	924	17	38	979
Pepper	ground	800	100	100	1000
Plate	empty	307	38	39	384
	with-food	400	50	50	500
Bowl	empty	440	30	30	500
	with-whisked-egg	400	50	50	500
Total		7729	794	920	9843

Test Video. A 90-second omelette-making video (1920×1080, 30 FPS, 2,702 frames), sampled at 2 FPS to yield 193 frames.

Feature Extraction. For supervised classifiers, CLIP ViT-B/32 embeddings (512-dim) are pre-computed and cached as `.npy` files. For the VLM pipeline, DINOv2-small embeddings (384-dim) are computed on-the-fly per sampled frame.

Preprocessing. All images are resized and normalized using ImageNet statistics.

(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). No augmentation is applied at inference.

4 Change Detection via Embedding Similarity

We maintain a rolling anchor and compute cosine similarity for each new frame:

$$\text{sim}(\mathbf{e}_{\text{anchor}}, \mathbf{e}_{\text{current}}) = \frac{\mathbf{e}_{\text{anchor}} \cdot \mathbf{e}_{\text{current}}}{\|\mathbf{e}_{\text{anchor}}\| \|\mathbf{e}_{\text{current}}\|} \quad (1)$$

If $\text{sim} < \tau$, the frame is flagged. The anchor updates *only* on confirmed transitions, preventing drift. At $\tau = 0.85$, the system flagged 100+ frames (camera wobble, subtitle changes). At $\tau = 0.70$, it captured meaningful changes, reducing invocations from 193 to $\sim 30\text{--}40$ ($\sim 80\%$ reduction).

DINOv2-small [1] (22M params, 384-dim CLS token) was selected for the VLM pipeline after benchmarks across 15 keyframes showed superior within-class vs. cross-class separation (within-class: 0.85–0.97, cross-class: 0.30–0.65), using $\tau = 0.70$. For the supervised pipeline, CLIP ViT-B/32 [4] (512-dim) embeddings were used with a higher threshold of $\tau = 0.90$, as CLIP’s text-aligned embedding space produces tighter similarity scores between visually similar frames.

5 Classification Approaches

5.1 Supervised Classifiers

Three classifiers were trained on CLIP ViT-B/32 [4] embeddings (512-dim):

- **SVM with RBF Kernel.** One-vs-rest, $C = 10.0$, $\gamma = \text{scale}$. Maps embeddings into a higher-dimensional space via the kernel trick for non-linear boundaries.
- **SVM with Linear Kernel.** $C = 1.0$, finding a hyperplane directly in the 512-dim space. Serves as an interpretable baseline.
- **Logistic Regression.** Multinomial with L2 regularization ($C = 1.0$), L-BFGS solver, 5,000 max iterations. Outputs calibrated probabilities used as confidence scores.

5.2 Unsupervised Neural Network: Deep Embedded Clustering (DEC)

DEC [8] uses a stacked autoencoder (512 $\rightarrow$ 1024 $\rightarrow$ 512 $\rightarrow$ 128) to learn a latent representation, then refines cluster assignments via a Student’s t -distribution:

$$q_{ij} = \frac{(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|^2/\alpha)^{-\frac{\alpha+1}{2}}} \quad (2)$$

with $\alpha = 1$ and learnable centroids initialized by K -Means. The DEC loss minimizes $\text{KL}(P\|Q)$. Pretrained 100 epochs, fine-tuned 150 epochs, 12 clusters. Cluster-to-label mapping via Hungarian algorithm.

5.3 Zero-Shot: Vision-Language Model (VLM)

The VLM approach requires *no training data*, instead using autoregressive context accumulated across the video.

- **Schema and Forward-Only Constraint.** An LLM (Gemini 2.5 Flash [2]) generates objects, ordered states, and process steps from the task description. States are ordered chronologically, and a **forward-only constraint** is enforced: an object’s state can only advance through the list, never regress. This is the key mechanism that eliminates state flickering.
- **Scene Inventory and Position Table.** On the first frame, the VLM identifies every visible instance, assigns unique IDs (e.g., `Egg_1`), and records spatial descriptions. This creates an autoregressive **position table** that updates as objects move.

- **Autoregressive Tracking.** Each flagged frame sends the VLM: (1) anchor and current frames, (2) current position table, (3) recent memory bank history (last 10 transitions), (4) process steps with current step, (5) possible states per class. This full temporal context prevents redundant or contradictory detections.
- **Multi-Instance Tracking.** Spatial descriptions disambiguate duplicates. If `Egg_1` is in the pan and a new egg appears in the bowl, the system registers `Egg_2`.
- **Fallback.** Primary: Gemini 2.5 Flash API (500 req/day). On rate limit, automatic fallback to Gemma 4 E4B [3] running locally in bfloat16 (~ 3.5 GB VRAM) alongside DINOv2-small on an 8 GB GPU.

6 Results

6.1 Classification Accuracy

Table 2: Test-set classification performance.

Model	Accuracy	F1 Macro	F1 Wtd.	Type
SVM RBF ($C=10$)	0.855	0.822	0.856	Supervised
SVM Linear ($C=1$)	0.845	0.810	0.846	Supervised
Logistic Regression	0.833	0.796	0.835	Supervised
DEC	0.612	0.342	0.542	Unsup. Neural Net

SVM RBF achieves the best per-frame accuracy. The three supervised methods cluster within 2.1 percentage points, indicating the CLIP embedding space is already well-structured. All show cross-validation std < 0.003 across 5 folds.

6.2 Per-Class Analysis

Table 3: Per-class F1 on test set (selected classes).

Class	SVM RBF	SVM Lin.	LogReg	DEC
eggs/raw	1.000	1.000	1.000	0.889
eggs/omelette	0.993	0.991	0.982	0.808
pepper/ground	1.000	1.000	1.000	0.782
oil/general	1.000	1.000	1.000	0.000
eggs/whisked	0.204	0.216	0.231	0.035
bowl/w-whisked-egg	0.267	0.240	0.188	0.466
pan/with-food	0.681	0.619	0.594	0.000
plate/with-food	0.743	0.699	0.667	0.000
Macro avg	0.822	0.810	0.796	0.342

Classes with distinct visual signatures achieve perfect F1. The weakest classes—`eggs/whisked` ($F1 < 0.27$) and `bowl/with-whisked-egg`—suffer from high visual overlap with neighboring states. DEC outperforms supervised models on `bowl/with-whisked-egg` (0.466 vs. ≤ 0.267), likely because its clusters are not biased by label distribution.

6.3 Memory Bank Results and the Flickering Problem

The memory bank is the primary pipeline deliverable. However, **per-frame classification without temporal context produces state flickering**, rendering some supervised memory

banks unreliable despite high per-frame accuracy. Table 4 quantifies this.

Table 4: State flickering analysis: egg-state oscillations in each model’s memory bank.

Model	Total Trans.	Egg Trans.	Egg Flickers	Egg State Sequence (abridged)
SVM RBF	16	6	4	wh→om→wh→om→wh→om
SVM Linear	12	4	2	wh→om→wh→om
LogReg	9	1	0	wh→om
DEC	11	5	4	om→wh→om→wh→om
VLM	9	6	0	cracked→beaten→cooking→flat→folded

wh = whisked, *om* = omelette. A “flicker” is a backward transition to a previously held state.

6.4 Memory Bank Comparison

Table 5: Memory bank statistics for the 90-second test video.

Model	Trans.	Flickers	Obj.	Avg Conf.	Time
SVM RBF	16	4	5	0.608	<1 min
SVM Linear	12	2	6	0.589	<1 min
LogReg	9	0	5	0.761	<1 min
DEC	11	4	5	0.099	<1 min
VLM	9	0	5	high	5 min (API) 30 min (local)

Ground truth contains ~ 10 meaningful transitions. The VLM’s 9 (0 flickers) and LogReg’s 9 (0 egg flickers) are closest to ground truth. SVM RBF’s 16 transitions include 4 egg oscillations despite best per-frame accuracy.

6.5 Memory Bank Snapshots: Classical Models

Tables 6–9 show excerpts from each classical model’s memory bank, focusing on flickering regions. Red highlights backward transitions (flickers).

Table 6: SVM RBF — egg and bowl flickering (4 egg flickers, 2 bowl flickers).

Time	Obj	New State	Prev State	Cf
0:01	eggs	whisked	–	64%
0:07	bowl	w-whisked-egg	empty	64%
0:14	bowl	empty	w-whisked-egg	32%
0:25	bowl	w-whisked-egg	empty	64%
0:37	eggs	omelette	whisked	64%
0:49	eggs	whisked	omelette	64%
1:00	eggs	omelette	whisked	64%
1:14	eggs	whisked	omelette	46%
1:15	bowl	empty	w-whisked-egg	65%
1:17	eggs	omelette	whisked	64%
1:19	bowl	w-whisked-egg	empty	64%
<i>16 total transitions, 5 omitted (pan, oil, butter)</i>				

Table 7: SVM Linear — egg and pan flickering (2 egg flickers, 1 pan flicker).

Time	Obj	New State	Prev State	Cf
0:27	pan	with-food	empty	64%
0:30	pan	empty	with-food	71%
0:35	eggs	whisked	–	67%
0:37	eggs	omelette	whisked	64%
0:44	eggs	whisked	omelette	36%
0:45	eggs	omelette	whisked	39%
0:52	pan	with-food	empty	45%
<i>12 total transitions, 5 omitted (plate, bowl, oil, butter)</i>				

Table 8: Logistic Regression — cleanest supervised bank. No egg flickers, pan oscillates once.

Time	Obj	New State	Prev State	Cf
0:00	bowl	w-whisked-egg	–	99%
0:11	eggs	whisked	–	89%
0:14	pan	empty	–	45%
0:16	pan	with-food	empty	97%
0:30	pan	empty	with-food	72%
0:31	oil	general	–	77%
0:32	pan	with-food	empty	98%
0:40	eggs	omelette	whisked	66%
1:14	butter	solid	–	41%
<i>9 total transitions (complete)</i>				

Table 9: DEC (Unsupervised Neural Network) — egg flickering with near-random confidence ($\sim 9\text{--}11\%$).

Time	Obj	New State	Prev State	Cf
0:00	eggs	omelette	–	9%
0:35	bowl	empty	w-whisked-egg	10%
0:41	eggs	whisked	omelette	11%
0:44	eggs	omelette	whisked	9%
1:16	eggs	whisked	omelette	13%
1:17	pepper	ground	–	12%
1:19	bowl	w-whisked-egg	empty	10%
1:20	eggs	omelette	whisked	8%
<i>11 total transitions, 3 omitted (butter, plate)</i>				

The pattern is consistent: **all four classical models flicker on the same ambiguous states**—primarily **whisked** $\leftrightarrow$ **omelette** for eggs and **empty** $\leftrightarrow$ **with-food** for pan. DEC additionally suffers from near-random confidence ($\sim 9\text{--}11\%$), making even its correct predictions unreliable. Logistic Regression produces the cleanest supervised bank (9 transitions, 0 egg flickers) but still oscillates on pan state. All classical models classify each frame in isolation with no memory of prior predictions.

6.6 VLM Memory Bank: Zero Flickering

The VLM produces **zero flickers** through three mechanisms: (1) the **forward-only constraint** prevents backward transitions, (2) **memory bank history** in the prompt gives temporal context, and (3) **two-frame comparison** forces the VLM to identify *what changed* rather than classify in isolation.

Table 10: VLM memory bank (complete): 9 transitions, 0 flickers, no training data.

Time	Object	New State	Prev State	Frame
0:01	Egg_1	cracked_in_bowl	whole_in_shell	42
0:10	Egg_1	beaten_liquid	cracked_in_bowl	322
0:21	Frying Pan_1	oiled_hot	empty_clean_cold	630
0:25	Frying Pan_1	with_liquid_eggs	oiled_hot	770
0:35	Egg_1	cooking_solidifying	beaten_liquid	1064
0:35	Frying Pan_1	with_cooking_om.	with_liquid_eggs	1064
1:17	Egg_1	cooked_flat	cooking_solidifying	2310
1:18	Egg_1	folded_omelette	cooked_flat	2338

Table 11: VLM position table (final snapshot).

Instance	Final State	Location
Egg_1	folded_omelette	on Plate_1
Frying Pan_1	with_cooking_omelette	on stovetop
Whisk_1	clean_dry	held above Pan_1
Spatula_1	clean_dry	in Pan_1
Plate_1	empty_clean	next to Pan_1

The egg lifecycle is tracked across 6 transitions with timestamps within 2–3s of ground truth. Limitations: the second egg at video start was not detected as a separate instance, cheese at ~70s was missed (absent from schema), and Plate_1’s final state was not updated to reflect the plated omelette.

7 Discussion

7.1 The Context Gap

The central finding is that **per-frame accuracy does not predict memory bank quality**. SVM RBF (best accuracy, 85.5%) produces a noisy memory bank (16 transitions, 4 egg flickers). Per-frame classifiers have no concept of “the egg was already cooking” and oscillate whenever the visual signal is ambiguous.

The VLM solves this via context injection: position table, history, and process steps provide a running narrative. For temporal tracking, **context awareness matters more than per-frame accuracy**.

7.2 Tradeoffs

Supervised classifiers are fast (<1 min) but require 9,843 labeled images and produce noisier memory banks. The VLM requires no training data and produces clean banks with multi-instance support, but is slower (~5 min via Gemini API, ~30 min via local Gemma 4). A hybrid—SVM for initial classification, VLM for ambiguous frames—could combine both strengths.

7.3 Limitations and Future Work

- **Schema completeness.** The LLM missed “cheese” as a trackable object. A human-in-the-loop schema review could address this.
- **Generalization.** All experiments use omelette-making. Evaluation on diverse procedural tasks (assembly, chemistry) would assess domain transfer.

- **Post-hoc smoothing.** Sliding-window majority voting could reduce supervised flickering, though it cannot match VLM-level semantic reasoning.
- **Per-object embeddings.** Currently, change detection operates on full-frame embeddings, which conflate changes across all objects. Ideally, an object detector (e.g., Grounding DINO) would first produce bounding boxes for each tracked object, and embedding similarity would be computed per-crop. This would enable detecting state changes for individual objects independently—e.g., noticing the egg changed state while the pan did not—reducing false positives and enabling finer-grained temporal tracking.

8 Conclusion

We presented a two-stage pipeline for temporal object state change detection. The embedding-based front-end reduces processing by $\sim 80\%$. Among supervised methods, SVM RBF achieves the best per-frame accuracy (85.5%) but suffers state flickering. The zero-shot VLM, leveraging autoregressive context and forward-only constraints, produces the most reliable memory bank with zero flickering and no training data. Our key finding: **temporal context awareness—not per-frame accuracy—is the critical factor for producing usable state transition logs from procedural videos**. Code and results are in the accompanying repository.

References

- [1] Oquab, M., et al. DINOv2: Learning Robust Visual Features without Supervision. *TMLR*, 2024.
- [2] Google DeepMind. Gemini: A Family of Highly Capable Multimodal Models. *arXiv:2312.11805*, 2024.
- [3] Google DeepMind. Gemma 4: Open Models Built from Gemini 3 Research. *Technical Report*, April 2026.
- [4] Radford, A., et al. Learning Transferable Visual Models From Natural Language Supervision. *ICML*, 2021.
- [5] Grauman, K., et al. Ego4D: Around the World in 3,000 Hours of Egocentric Video. *CVPR*, 2022.
- [6] Souček, T., et al. Multi-modal Object State Change Detection. *ECCV*, 2024.
- [7] Bardes, A., et al. V-JEPA: Latent Video Prediction for Visual Representation Learning. *arXiv:2404.16930*, 2024.
- [8] Xie, J., Girshick, R., Farhadi, A. Unsupervised Deep Embedding for Clustering Analysis. *ICML*, 2016.